CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture - 5

Contemporary Processor Architectures for Embedded Systems

- ARM (Cortex-A and Cortex-M)
  - Raspberry Pi ($30), Nvidia Jetson ($1000), BeagleBoard, Tinker Board, etc.
- AVR microcontrollers
  - Arduino series of boards
- Intel (Atom, Quark)
  - Edison, Galileo, Up Squared boards (Pentium, Celeron)
- RISC-V
  - SiFive, HiFive, Star64, ESP32 (C and H series boards)

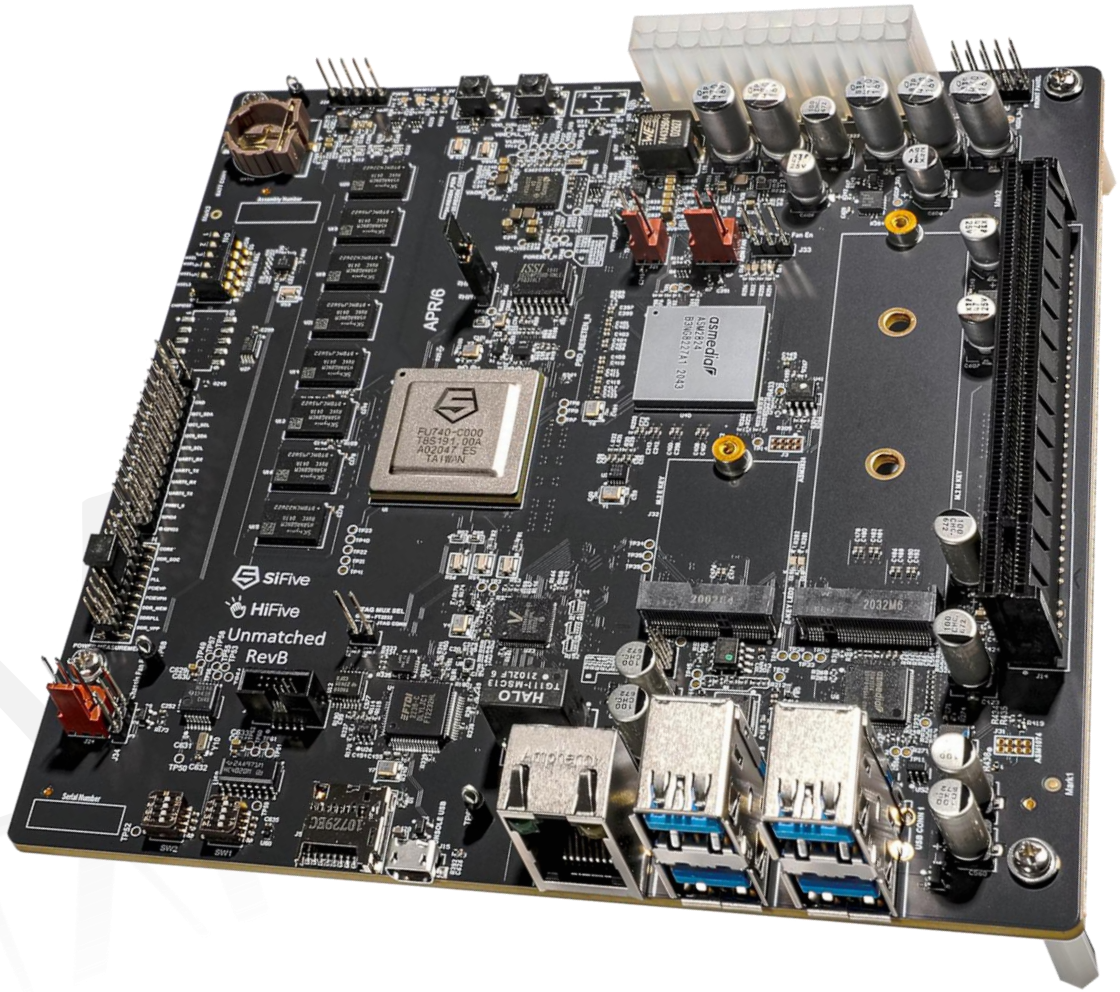
SiFive IP Cores

- **SiFive** Core IP portfolio comprises four distinct families
  - All SiFive processors are based upon the RISC-V ISA.

  - SiFive Performance: High-performance application processors (e.g. P870, P550)
    - Out-of-order, superscalar, vector units, clusters of cores, MMU
  - SiFive Essential: Area-optimized, low-power embedded 64- and 32-bit microprocessors (e.g. U74, U6)
    - In-order, superscalar design, MMU, for use in IoT devices, real-time control applications
  - SiFive Intelligence: Vector processors designed for modern compute requirements and artificial intelligence (e.g. X390, X380)
    - In-order, superscalar, vector processor
  - SiFive Automotive: Optimized for the specific needs of the automotive industry (e.g. S7-AD)
    - In-order, superscalar, additional safety features for memories, Automotive Safety Integrity Level

HiFive Unmatched RevB

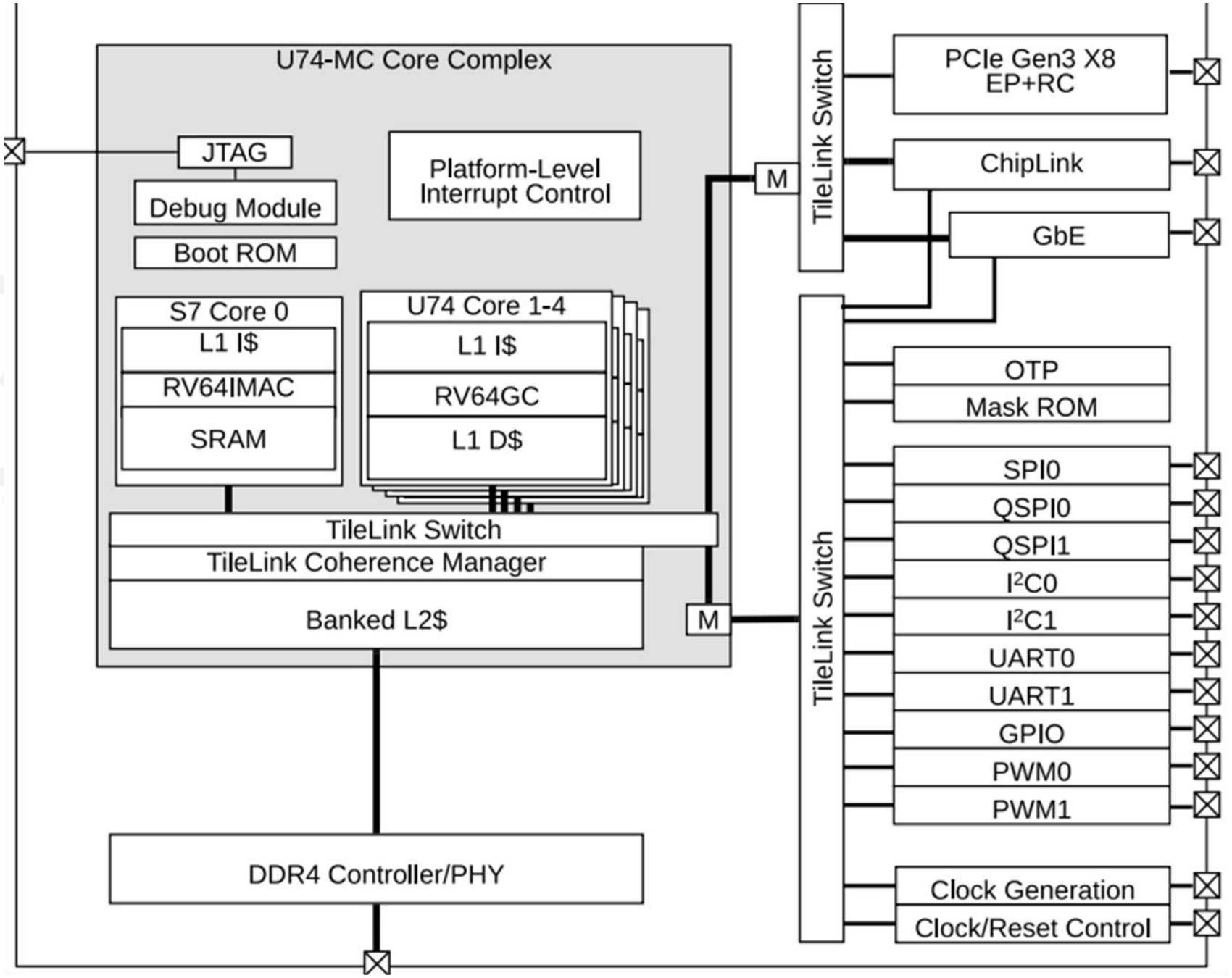
- Freedom 740 SoC
  - Linux capable
  - Four application cores, 64-bit dual-issue, superscalar RISC-V processors (U74)
  - A 5th core, monitor core, is available for real-time applications (S71)



Freedom 740

5 Harts
(**HAR**dware **T**hread)
An independent, physical or logical execution unit that fetches and executes instructions autonomously, possessing its own unique architectural state (Program Counter and registers)

A HART is an independent execution unit inside a processor that can run its own instructions.



SiFive U74 processor

Feature	Description
Architecture	8-stage, dual-issue, in-order pipelined processor
ISA	RV64GC_Zba_Zbb_Sscofpmf RV64I base ISA, as well as the Multiply (M), Atomic (A), Single- and Double-Precision Floating Point (F and D), Compressed (C), CSR Instructions (Zicsr), Instruction-Fetch Fence (Zifencei), Address Calculation (Zba), Basic Bit Manipulation (Zbb), and Count Overflow and Mode-Based Filtering (Sscofpmf) RISC-V extensions
Modes	Machine mode (M), supervisor mode (S), user mode (U)
L1 Instruction Cache	32 KiB 4-way instruction cache
L1 Data Cache	32 KiB 8-way data cache
L2 Cache	2 MiB 16-way L2 cache with 4 banks
ECC Support	SECDED (Single error correction double error detection) on the data cache and L2 cache.
Physical Memory Protection	8 regions with a granularity of 4096 bytes
Memory Management Unit	Sv39 virtual memory support with fully-associative 40-entry L1 Data and Instruction TLBs, and a direct-mapped 512-entry L2 TLB.

SiFive U74 processor

In the SiFive U74, privilege modes define different levels of access and control a program can have over the processor and system.

They are security levels that control:

Who can access hardware
Who can execute certain instructions
Who controls the system

- Privileged Modes
  - U74 supports different execution modes, providing three levels of privilege:
    - Machine (M) Full control over the processor
      Can access all instructions and registers
      - This is the highest privilege level and the only mandatory mode for all RISC-V harts
      - Handles hardware initialization (memory protection, peripheral controllers), and exceptions
    - Supervisor (S)
      - This is the middle tier, designed for running an Operating System kernel (like Linux)
      - Manages system resources, scheduling, and I/O for applications
    - User (U) (lowest level)
      - Provides a restricted environment so that a single application crash cannot take down the entire system
      - Attempting to access hardware registers or another program's memory will trigger an exception, causing the CPU to jump to a higher mode (S or M) to handle the violation

Control and Status Registers

- In RISC-V, Control and Status Registers (CSRs) are special registers used to control the processor, report status, handle exceptions/interrupts and monitor performance.
  - We can control certain aspects of the processor by flipping bits in the CSR register and we can get the status of the machine simply by reading one of these registers
- Unlike general-purpose registers (x0–x31), CSRs reside in their own 12-bit address space and require special instructions to access.
- Each CSR is of width equal to XLEN i.e. for U74 it is of length 64.
- The address of the CSR encodes the privilege level and the read/write permission
  - [11:10] defines the read/write permission and [9:8] the lowest privilege level that can access it
  - E.g. mstatus is at address 0x300, stsatus is at address 0x100 and ustatus is at address 0x000
- They are accessed via dedicated CSR instructions (CSRRW, CSRRS, CSRRC, and their immediate variants)

CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture - 6

SiFive U74 processor

- RV64GC
  - the abbreviation G is for the IMAFD_Zicsr_Zifencei combination of instruction-set extensions. C is for Compressed instructions of 16-bits
  - there are new instructions in RV64GC due to the 64-bit architecture, and support for multiplication, atomic operations, floating-point, and compressed instructions.
- The processor is a 64-bit processor i.e. it allows 64-bit operations
- The instruction size is still 32 bits but 16 bits for compressed instructions
- The number of registers include 32 general purpose registers (x0 - x31) GPR, 32 floating point registers (f0- f31) plus a maximum of 4096 control and status registers CSR
- All registers including PC are 64-bit wide.

RV64GC Instruction Set

RV64I Instructions

Instruction	Example	Meaning	Comments
Add	`add x5, x6, x7`	`x5 = x6 + x7`	Three register operands; add
Subtract	`sub x5, x6, x7`	`x5 = x6 - x7`	Three register operands; subtract
Add immediate	`addi x5, x6, 20`	`x5 = x6 + 20`	Used to add constants

- Same as the previous RV32I instruction set, however, all operations are now 64-bit
- Because instruction size is 32-bits the immediate is still a 12 bit signed number which is extended to 64-bits
- The opcode of these instructions plus the funct3 and funct7 values are same as RV32I

RV64GC ISA

Additional Arithmetic/Shift Instructions

- 32-bit processing on 64-bit architecture
  - ADDW and SUBW are RV64I instructions that are defined analogously to ADD and SUB but operate on 32-bit values and produce signed 32-bit results.
    - Overflows are ignored, and the low 32-bits of the result is sign-extended to 64-bits and written to the destination register.
  - ADDIW is an RV64I instruction that adds the sign-extended 12-bit immediate to register rs1 and produces the proper sign-extension of a 32-bit result in rd.
    - Overflows are ignored and the result is the low 32 bits of the result sign-extended to 64 bits.
  - SLLW, SRLW, SRAW, SLLIW, SRLIW, SRAIW: Shifts lower 32 bits of a 64-bit register to the left or right and sign-extends the 32-bit result
    - useful for 32-bit integer arithmetic in a 64-bit environment

RV64GC ISA

- Modification in Shift Instructions
  - The existing shift instructions SLLI, SRLI and SRAI are modified
    - Since the register length is now 64 bits the shift amount is now a 6-bit number instead of a 5-bit number

RV64GC ISA - Data Transfer Instructions

Doubleword = 2 × word = 64 bits (8 bytes) in RV64.
It matches the register size in a 64-bit processor.

Instruction	Example	Meaning	Comments
Load doubleword	`ld x5, 40(x6)`	`x5 = Memory[x6 + 40]`	Doubleword from memory to register
Store doubleword	`sd x5, 40(x6)`	`Memory[x6 + 40] = x5`	Doubleword from register to memory
Load word, unsigned	`lwu x5, 40(x6)`	`x5 = Memory[x6 + 40]`	Unsigned word from memory to register
Load upper immediate	`lui x5, 0x12345`	`x5 = 0x0000000012345000` sign extended and shifted left (64 bits log now)	Loads 20-bit constant shifted left 12 bits

16-bit number is sign-extended to 64 bits

- Three more instructions are added for the 64-bit operations
  - ld load double
  - sd store double
  - lwu load word unsigned just like previously we had lhu or lbu
- Modification in lui
  - lui load upper immediate is going to load the 20-bit immediate in the register and the most significand 16 bits are sign extended while the lower 12 bits are all 0s.

RV64I Additional Instructions

RV64I Base Instruction Set (in addition to RV32I)

imm[11:0]		rs1	110	rd	0000011	LWU
imm[11:0]		rs1	011	rd	0000011	LD
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	SD
000000	shamt	rs1	001	rd	0010011	SLLI
000000	shamt	rs1	101	rd	0010011	SRLI
010000	shamt	rs1	101	rd	0010011	SRAI
imm[11:0]		rs1	000	rd	0011011	ADDIW
0000000	shamt	rs1	001	rd	0011011	SLLIW
0000000	shamt	rs1	101	rd	0011011	SRLIW
0100000	shamt	rs1	101	rd	0011011	SRAIW
0000000	rs2	rs1	000	rd	0111011	ADDW
0100000	rs2	rs1	000	rd	0111011	SUBW
0000000	rs2	rs1	001	rd	0111011	SLLW
0000000	rs2	rs1	101	rd	0111011	SRLW
0100000	rs2	rs1	101	rd	0111011	SRAW

RV64GC ISA

Additional Arithmetic Instructions

- Multiplication and Division Instructions
  - MUL rd, rs1, rs2 64-bit x 64-bit signed multiplication (lower 64 bits of the result placed in rd)
  - MULH rd, rs1, rs2 64-bit x 64-bit signed multiplication (upper 64 bits of the result placed in rd)
  - MULHU rd, rs1, rs2 64-bit x 64-bit unsigned multiplication (upper 64 bits of the result placed in rd)
  - MULHSU rd, rs1, rs2 64-bit signed (rs1) x 64-bit unsigned (rs2) (upper 64 bits of the result placed in rd)
  - MULW rd, rs1, rs2 32-bit x 32-bit signed multiplication, sign-extend the result to 64 bits

$$\text{dividend} = \text{divisor} \times \text{quotient} + \text{remainder}$$

  - DIV rd, rs1, rs2 64-bit signed division rounding towards zero
  - DIVU rd, rs1, rs2 64-bit unsigned division
  - REM and REMU provide the remainder of the corresponding division operation.
  - For REM, the sign of a nonzero result equals the sign of the dividend.

RV64GC ISA

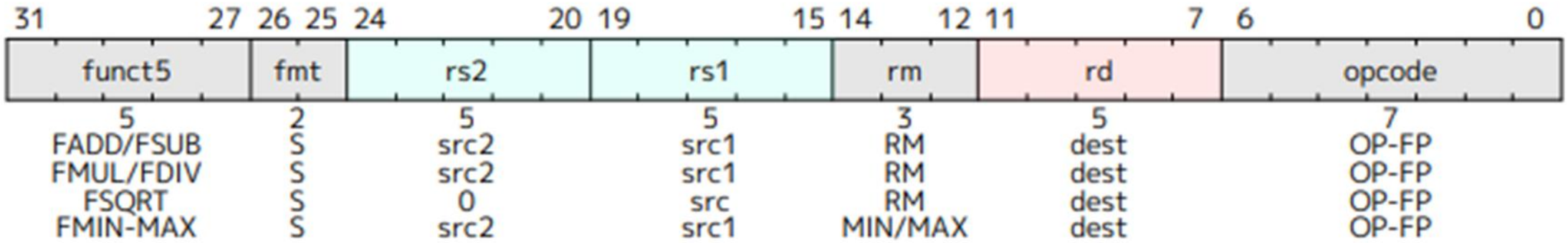
Additional Arithmetic Instructions

- Floating Point Arithmetic
  - FADD.S/D rd, rs1, rs2, rm Floating-point addition
  - FSUB.S/D rd, rs1, rs2, rm Floating-point subtraction
  - FMUL.S/D rd, rs1, rs2, rm Floating-point multiplication
  - FDIV.S/D rd, rs1, rs2, rm Floating-point division.
  - And many more

The 2-bit floating-point format field fmt defines the precision of the FP numbers

fmt field	Mnemonic	Meaning
00	S	32-bit single-precision
01	D	64-bit double-precision
10	H	16-bit half-precision
11	Q	128-bit quad-precision

rounding mode = rm

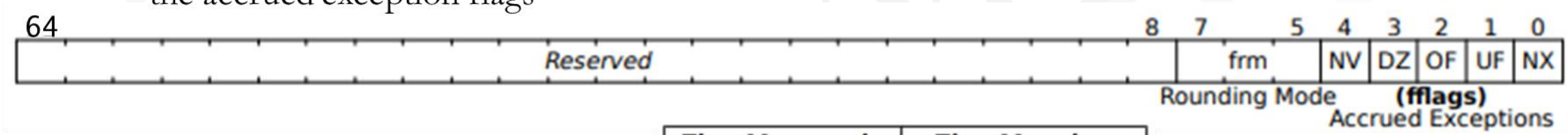


RV64GC ISA

- Rounding Modes (rm field)
  - Floating-point operations use either a static rounding mode encoded in the instruction, or a dynamic rounding mode held in frm

Mode	Mnemonic	Meaning
000	RNE	Round to Nearest, ties to Even.
001	RTZ	Round towards Zero.
010	RDN	Round Down (towards - ∞).
011	RUP	Round Up (towards + ∞).
100	RMM	Round to Nearest, ties to Max Magnitude.
101		*Invalid. Reserved for future use.*
110		*Invalid. Reserved for future use.*
111	DYN	selects dynamic rounding mode:

- Floating-Point Control and Status Register
  - The register selects the dynamic rounding mode for floating-point arithmetic operations and holds the accrued exception flags



Flag Mnemonic	Flag Meaning
NV	Invalid Operation
DZ	Divide by Zero
OF	Overflow
UF	Underflow
NX	Inexact

CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture - 7

RV64GC ISA

- Compressed Instructions
  - The C extension provides 16-bit compressed versions of frequently used instructions, reducing code size and improving efficiency.
  - 16-bit instructions can be freely intermixed with 32-bit instructions

Compressed instruction would be used when

- the immediate or address offset is small, or
- one of the registers is the zero register (x0), the link register (x1), or the stack pointer (x2), or
- the destination register and the first source register are identical, or
- the registers used are the 8 most popular ones. For integers (x8-x15) for floating point (f8-f15)

RV64GC ISA

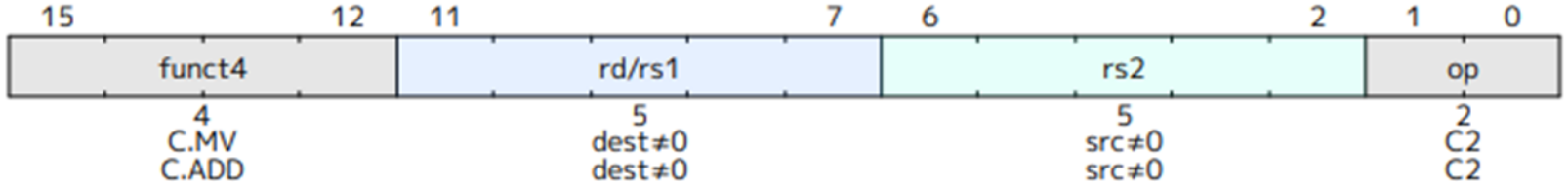
- Compressed Instructions

Format	Meaning	15 14 13	12	11 10	9 8 7	6 5	4 3 2	1 0
CR	Register	funct4		rd/rs1		rs2		op
CI	Immediate	funct3	imm	rd/rs1		imm		op
CSS	Stack-relative Store	funct3	imm			rs2		op
CIW	Wide Immediate	funct3	imm				rd′	op
CL	Load	funct3	imm		rs1′	imm	rd′	op
CS	Store	funct3	imm		rs1′	imm	rs2′	op
CA	Arithmetic	funct6			rd′/rs1′	funct2	rs2′	op
CB	Branch/Arithmetic	funct3	offset		rd′/rs1′	offset		op
CJ	Jump	funct3	jump target					op

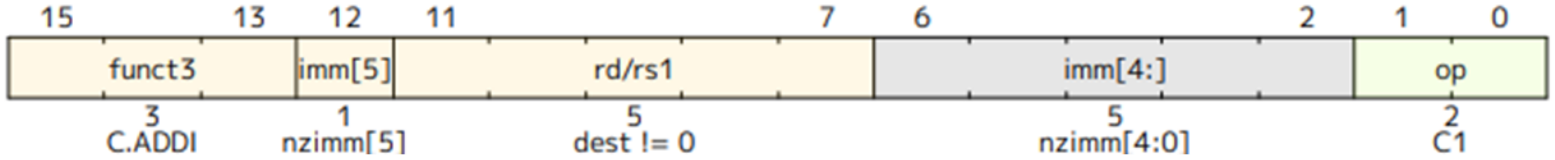
RV64GC ISA

Additional Arithmetic Instructions

- Compressed Instructions
  - C.ADD adds the values in registers rd and rs2 and writes the result to register rd. C.ADD expands into add rd, rd, rs2. C.ADD is only valid when rs2≠x0



  - C.ADDI adds the non-zero sign-extended 6-bit immediate to the value in register rd then writes the result to rd. C.ADDI expands into addi rd, rd, imm. C.ADDI is only valid when rd≠x0 and imm≠0.



C to RV64GC assembly

- Let's take a simple C program and transform it into RISC-V assembly for RV64GC
- RV64GC (32 bit instructions)

```
long long int add(long long int x, long long int y){
    long long int z;
    z = x + y;
    return z;
}
```

```
    add a0, a0, a1          # Put sum of a0 and a1 in a0
    ret
```

C to RV64GC assembly

- Let's take a simple C statement and transform it into RISC-V assembly for RV64GC
- RV64GC (16 bit instructions)

Compressed instructions could be used for load (ld) and store (sd) as long as the immediate value is a number divisible by 8 and can be stored in 8-bits.
We don't store the last three bits so that's why it should be divisible by 8. Actually, immediate has a space of 5 bits.

```
// Arr is an array of doublewords with
// base address in x20

Arr[10] = Arr[1]+Arr[2];
```

For loads/stores, registers for rs1 and rd can only be x8 to x15 where x8 is encoded as 000

```
c.mv x8, x20          # if loading from memory first use a register that should be from x8-x15
c.ld x9, 8(x8)        # Load into x9 from memory location which is 8 bytes from base address
c.ld x10, 16(x8)      # Load into x10 from memory location which is 16 bytes from base address
c.add x9, x10         # x9 = x9 + x10
c.sd x9, 80(x8)       # Store x9 to Arr[10]
```

8 x 10 = 80

lw,sw for 4B disatnce storage
ld,sd for 8B distance strage

C to RV64GC assembly

- Let's take a simple C statement and transform it into RISC-V assembly for RV64GC
- RV64GC (16 bit instructions)

```
// Arr is an array of words with
// base address in x20

Arr[10] = Arr[1] + Arr[2] + 20;
```

Compressed instructions could be used for load (lw) and store (sw) as long as the immediate value is a number divisible by 4 and can be stored in 7-bits.
We don't store the last two bits so that's why it should be divisible by 4. Actually, immediate has a space of 5 bits.

```
c.mv x8, x20          # if loading from memory first use a register that should be from x8-x15
c.lw x9, 4(x8)        # Load into x9 from memory location which is 4 bytes from base address
c.lw x10, 8(x8)       # Load into x10 from memory location which is 8 bytes from base address
c.add x9, x10         # x9 = x9 + x10
c.addi x9, 20         # immediate within -32 to 31
c.sw x9, 40(x8)       # Store x9 to Arr[10]
```

C to RV64GC assembly

- Code comparison

Arr[10] = Arr[1] + Arr[2] + 20;

compressed instruction are 2B each henc, 2 x 6 = 12
while other one is 4b each hence 4 x 6 = 20B

```
c.mv   x8, x20
c.lw   x9, 4(x8)
c.lw   x10, 8(x8)
c.add  x9, x10
c.addi x9, 20
c.sw   x9, 40(x8)
```

Total bytes = 12

```
lw   x9, 4(x20)
lw   x10, 8(x20)
add  x9, x9, x10
addi x9, x9, 20
sw   x9, 40(x20)
```

Total bytes = 20

C to RV64GC assembly

- Let's take a simple C program and transform it into RISC-V assembly for RV64GC
- RV64GC (double precision FP instructions)

```
int main(){
    double x = 10.5, y = 20.3;
    double z = x + y;
    return 0;
}
```

z is a memory location at 100 bytes from x20

```
.section .rodata        x and y are consatant here
x: .double 10.5
y: .double 20.3

.section .text
la   x5, x              # Load address of constant
fld  f10, 0(x5)         # Use f10 (in the f8-f15 range for compression)   fld loads double word
la   x6, y
fld  f11, 0(x6)                 # Use f11
fadd.d f12, f10, f11            # f12 = x + y
fsd  f12, 100(x20)              # Store z to memory
li   a0, 0                      # a0 (x10) is the standard return register
ret
```

d - double precision(32 bit float) if s = single precision (16 bit float)

CSR Instructions

- RISC-V defines six core instructions for interacting with Control and Status Registers (CSRs) as part of the "Zicsr" extension.
- They all perform an atomic read-modify-write on a CSR
  - CSRRW (Read and Write): Atomically swaps values between a CSR and an integer register. It reads the old value of the CSR into rd and writes the entire value of rs1 into the CSR.
  - CSRRS (Read and Set bits): Reads the old value of the CSR into rd and sets bits in the CSR that are high in rs1. (AND rs1)
  - CSRRC (Read and Clear bits): Reads the old value of the CSR into rd and clears bits in the CSR that are high in rs1. (AND ~rs1)
  - CSRRWI, CSRRSI, CSRRCI are immediate versions with an immediate field of 5 bits instead of rs1 and the operations remain the same

Examples

- If we want to write to a CSR without reading its value use x0 for rd

```
csrrw x0, mstatus, x1   # write only
```

- If we only want to read a CSR without writing to it use x0 for rs1

```
csrrs x5, mstatus, x0   # read mstatus
```

- Enable interrupts
  - Machine interrupt (mie) is the third bit of mstatus CSR

```
li    x1, 0x8       # MIE bit
csrrs     x0, mstatus, x1
```

RV64GC ISA

- Atomic Operations
  - The atomic-instruction extension contains instructions that atomically read-modify-write memory to support synchronization between multiple RISC-V harts (hardware threads) running in the same memory space.
  - The two forms of atomic instruction provided are load-reserved/store-conditional instructions and atomic fetch-and-op memory instructions
  - LR.D rd, (rs1) – Load Reserved (64-bit)
    - Loads a 64-bit doubleword from memory at rs1 and marks it as reserved.
  - SC.D rd, rs2, (rs1) – Store Conditional (64-bit)
    - Stores rs2 only if the reservation is still valid (atomic operation succeeds).
    - If successful, rd = 0; otherwise, rd = 1 (indicating another thread modified the memory).
    - If the reservation was invalidated the software must retry

RV64GC ISA

Suppose x10 contains the address of a shared counter, x5 = 3 (value to add):

amoadd.d x6, x5, (x10)
Memory at (x10) = 10 before instruction
After execution:
Memory (x10) = 10 + 3 = 13
x6 = 10 (original value before addition)

- Atomic Operations
  - Atomic Memory Operations
    - These **read-modify-write (RMW) operations** ensure that memory modifications are **atomic** and cannot be interrupted.
    - AMO.OP.SZ rd, rs2, (rs1)
      - SZ = W (32-bit) or D (64-bit)
      - OP = operation (ADD, SWAP, XOR, AND, OR, MIN, MAX, MINU, MAXU)
      - rd = old value (before modification)
      - rs2 = value used in the operation
      - (rs1) = memory location
    - AMOSWAP.D rd, rs2, (rs1) // 64-bit swap
    - AMOADD.D rd, rs2, (rs1) // 64-bit addition
      - Adds rs2 to memory at rs1, stores result back, returns old value, used for **counters**

RV64GC ISA

- Atomic Operations
  - Atomic Memory Operations